In [ ]:
```
#Name: Meenakshi Sangeeth
#Registration No: 240905630
#Section: PG
#Roll No: 34
```

In [ ]:
```
#For the MTCARS dataset, answer the specified questions with summarization and effective visuals.
```

In [3]:
```
import pandas as pd
```

In [4]:
```
dat_mtcars=pd.read_csv("mtcars.csv", index_col=0)
```

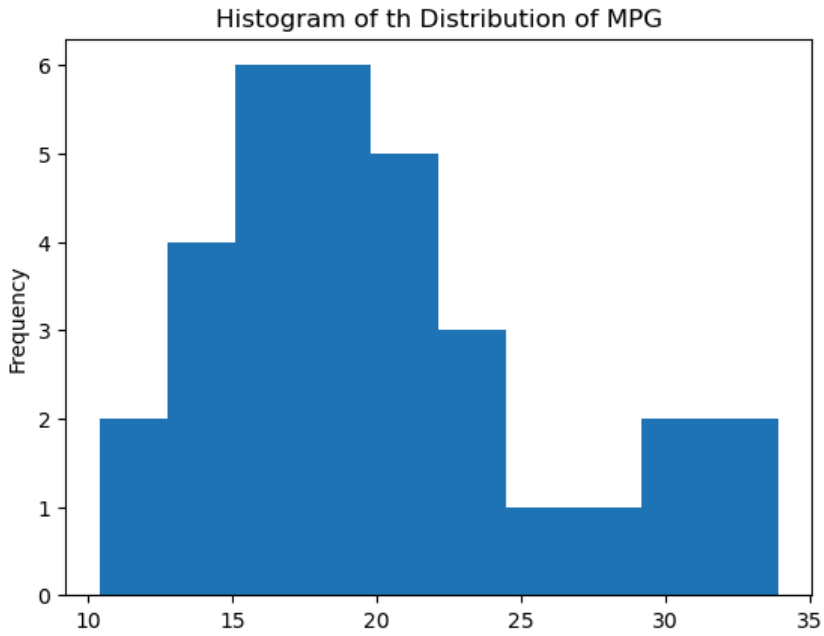
In [5]:
```
dat_mtcars
```

Out[5]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
model											
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

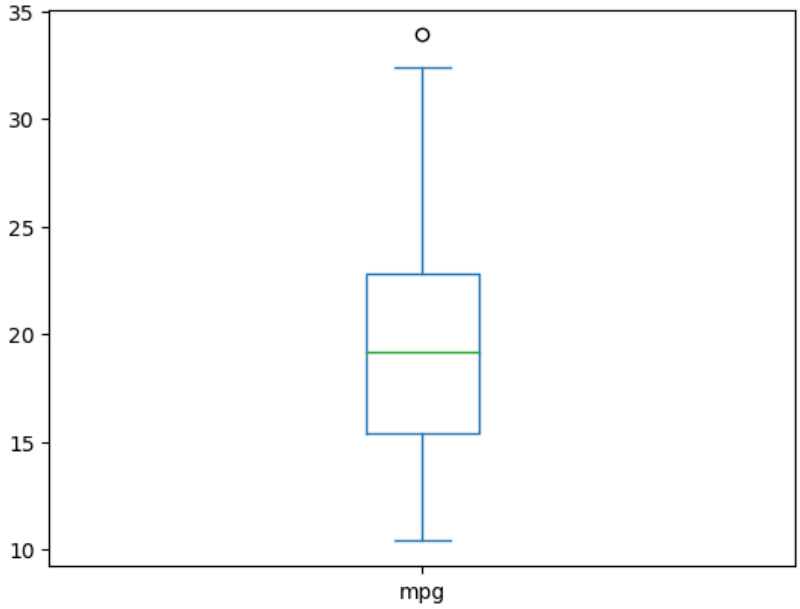
In [7]:
```
# 1. Explore the distribution of Fuel efficiency of the cars
# Remember to label both axis and put a title to the plot
dat_mtcars["mpg"].plot(kind='hist', bins=10, title="Histogram of th Distribution of MPG")
```

Out[7]:
```
<AxesSubplot:title={'center':'Histogram of th Distribution of MPG'}, ylabel='Frequency'>
```



In [8]:
```
# 1. Explore the distribution of Fuel efficiency of the cars
dat_mtcars["mpg"].plot(kind='box')
```

Out[8]:
```
<AxesSubplot:>
```



In [9]:
```
# 2. Find the cars with the best and worst fuel efficiency
max_mpg=dat_mtcars["mpg"].max()
dat_mtcars[dat_mtcars["mpg"]==max_mpg]
```

Out[9]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
model											
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.9	1	1	4	1

In [10]:
```
# 3. How many cars are manual vs automatic?
dat_mtcars["am"].nunique()
```

Out[10]: 2

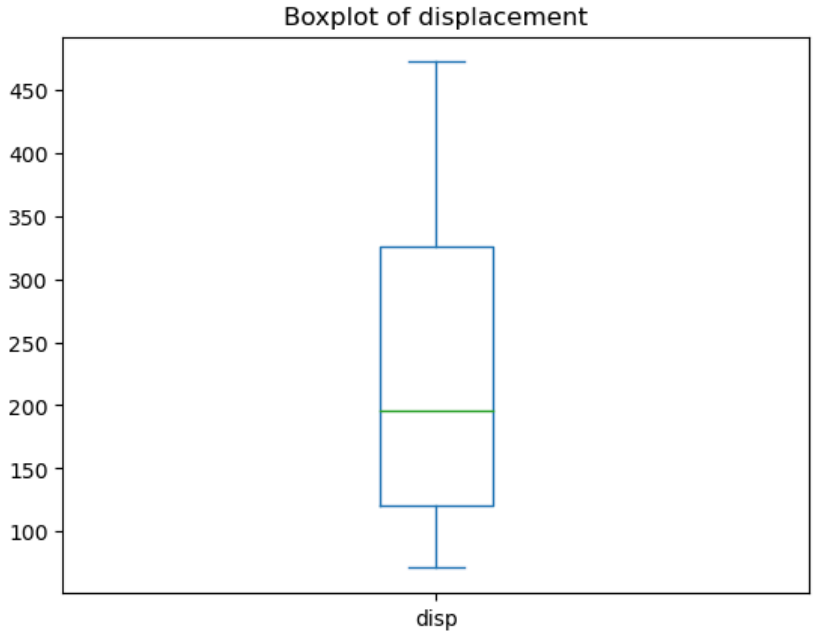
In [13]:
```python
# 4. Find the car with the worst horsepower
min_hp = dat_mtcars["hp"].min()
print("Car with worst horsepower:\n")
print(dat_mtcars[dat_mtcars["hp"]==min_hp]["hp"])
```

```
Car with worst horsepower:

model
Honda Civic    52
Name: hp, dtype: int64
```

In [15]:
```python
# 5. Find 5 number summary and draw the boxplot of displacement.
summary = dat_mtcars.describe().loc[['min','25%','50%','75%','max']]
print("Five number summary:\n")
print(summary)
dat_mtcars["disp"].plot(kind='box', title="Boxplot of displacement")
```

```
Five number summary:

        mpg  cyl     disp     hp   drat       wt     qsec   vs   am  gear  \
min  10.400  4.0   71.100   52.0  2.760  1.51300  14.5000  0.0  0.0   3.0
25%  15.425  4.0  120.825   96.5  3.080  2.58125  16.8925  0.0  0.0   3.0
50%  19.200  6.0  196.300  123.0  3.695  3.32500  17.7100  0.0  0.0   4.0
75%  22.800  8.0  326.000  180.0  3.920  3.61000  18.9000  1.0  1.0   4.0
max  33.900  8.0  472.000  335.0  4.930  5.42400  22.9000  1.0  1.0   5.0

     carb
min   1.0
25%   2.0
50%   2.0
75%   4.0
max   8.0
```

Out[15]:
```
<AxesSubplot:title={'center':'Boxplot of displacement'}>
```



In [17]:
```python
# 6. Which is the heaviest car? How many gears does it have?
wt_max = dat_mtcars["wt"].max()
print("Heaviest car:\n")
print(dat_mtcars[dat_mtcars["wt"]==wt_max]["gear"])
```

```
Heaviest car:

model
Lincoln Continental    3
Name: gear, dtype: int64
```

In [19]:

```
# 7. Which is the car with the best qsec?
qsec_max = dat_mtcars["qsec"].max()
print("Car with the best qsec:\n")
print(dat_mtcars[dat_mtcars["qsec"]==qsec_max]["qsec"])
```

```
Car with the best qsec:

model
Merc 230    22.9
Name: qsec, dtype: float64
```

In [11]:

```
# 8. What is average MPG for manual vs. automatic cars?
dat_man= dat_mtcars[dat_mtcars["am"]==1]["mpg"] # Manual cars are marked as 1
print("Average MPG of manual cars:",dat_man.mean())
dat_auto= dat_mtcars[dat_mtcars["am"]==0]["mpg"]
print("Average MPG of automatic cars:",dat_auto.mean())
```

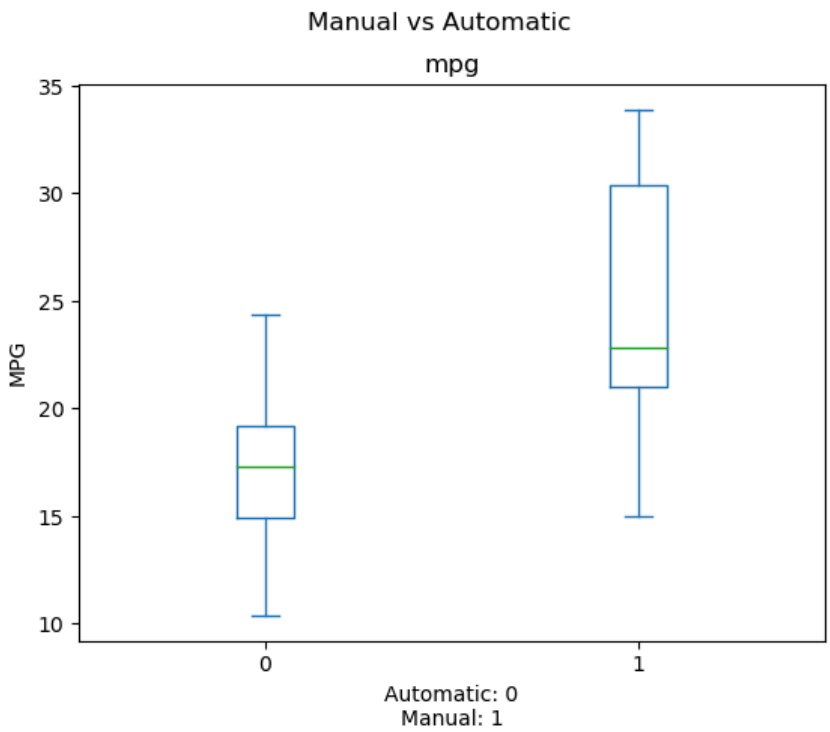
```
Average MPG of manual cars: 24.39230769230769
Average MPG of automatic cars: 17.147368421052633
```

In [27]:

```
# 9. Draw Side by Side box plot to understand the difference in fuel efficience of Manual vs Automatic c
x = dat_mtcars.plot(kind='box', column='mpg', by='am', title='Manual vs Automatic')
x[0].set_xlabel('Automatic: 0\nManual: 1')
x[0].set_ylabel('MPG')
```

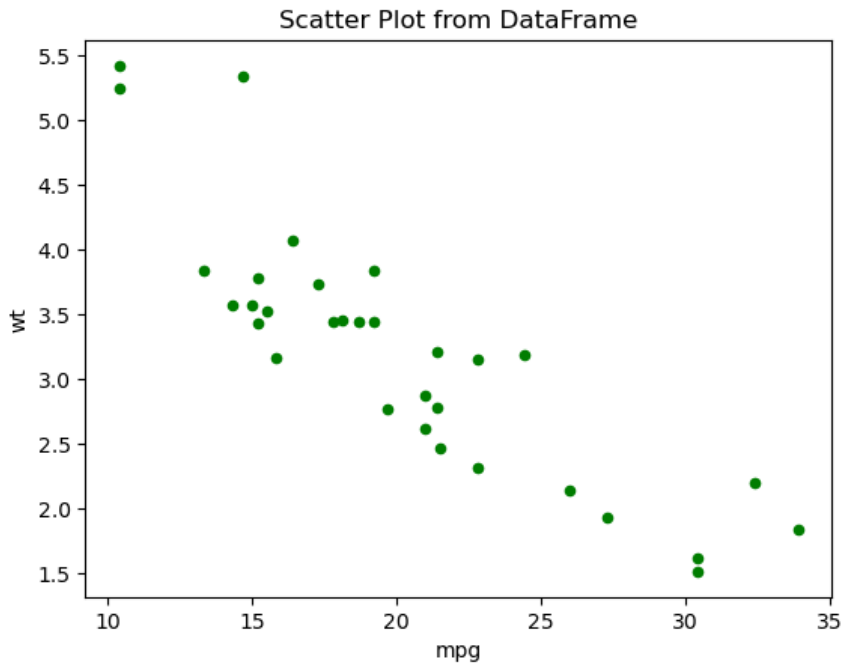
Out[27]:

```
Text(0, 0.5, 'MPG')
```



In [43]:
```
# 10. What is the relationship between the weight of the car and MPG?
dat_x= dat_mtcars[["mpg","wt"]]
dat_x= dat_x.reset_index(drop=True)
dat_x.plot(kind='scatter', x='mpg', y='wt', color='green', title='Scatter Plot from DataFrame')
```

Out[43]:
```
<AxesSubplot:title={'center':'Scatter Plot from DataFrame'}, xlabel='mpg', ylabel='wt'>
```



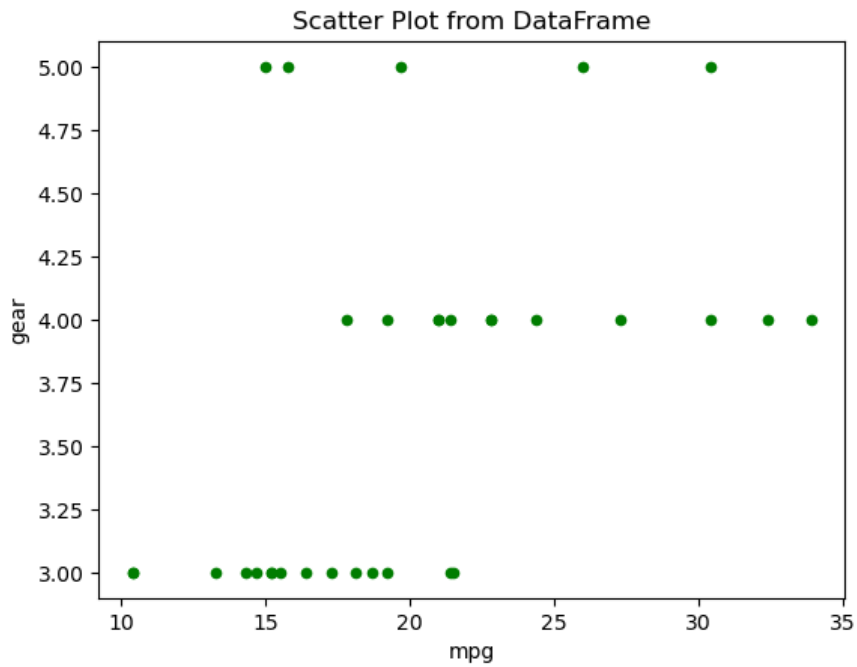
In [40]:
```
# 11. Categorize the cars based on the number of gears in the cars. How many cars are there in each type
print("Number of gear types:", dat_mtcars["gear"].nunique())
dat_mtcars["gear"].value_counts()
```

```
Number of gear types: 3
```

Out[40]:
```
3    15
4    12
5     5
Name: gear, dtype: int64
```

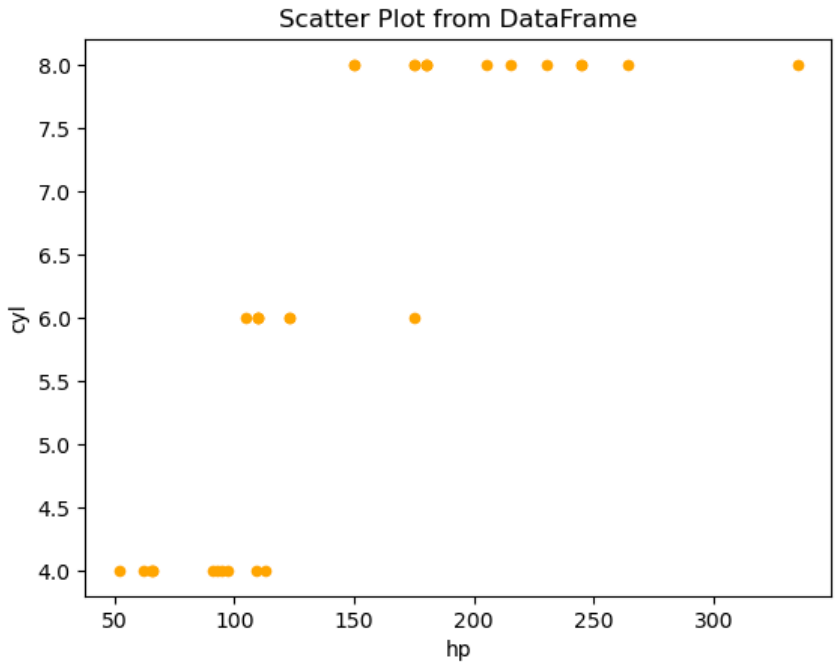
In [41]:
```
# 12. What is the relationship between fuel efficiency and the number of gears in the car?
dat_x= dat_mtcars[["mpg","gear"]]
dat_x.plot(kind='scatter', x='mpg', y='gear', color='green', title='Scatter Plot from DataFrame')
```

Out[41]:
```
<AxesSubplot:title={'center':'Scatter Plot from DataFrame'}, xlabel='mpg', ylabel='gear'>
```



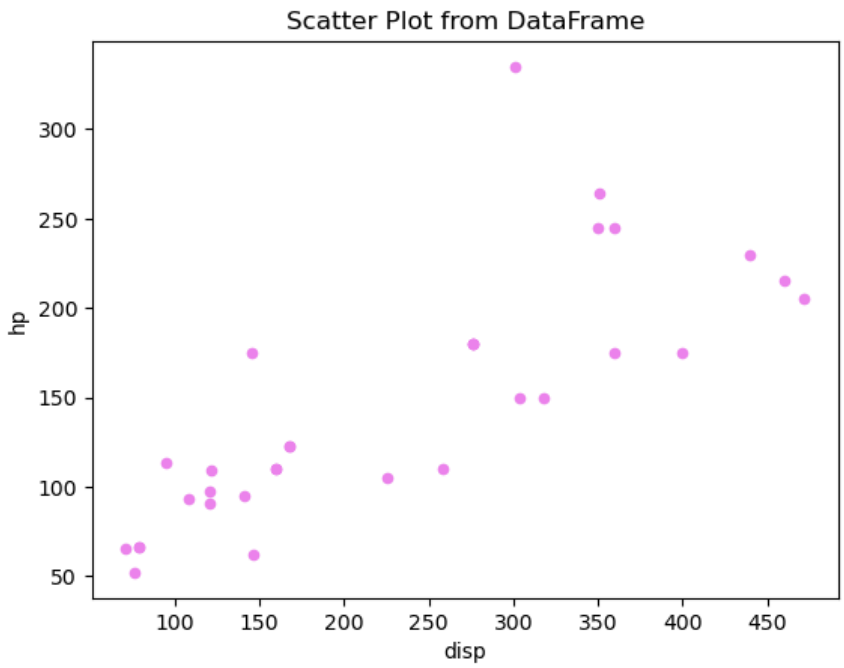
In [47]:
```
# 13. Explain the relationship between horse power and number of cylinders in the car.
dat_x= dat_mtcars[["hp","cyl"]]
dat_x.plot(kind='scatter', x='hp', y='cyl', color='orange', title='Scatter Plot from DataFrame')
```

Out[47]:
```
<AxesSubplot:title={'center':'Scatter Plot from DataFrame'}, xlabel='hp', ylabel='cyl'>
```



In [60]:
```
# 14. Explain the relationship between displacement and gross horse power.
dat_x= dat_mtcars[["disp","hp"]]
dat_x.plot(kind='scatter', x='disp', y='hp', color='violet', title='Scatter Plot from DataFrame')
```

Out[60]:
```
<AxesSubplot:title={'center':'Scatter Plot from DataFrame'}, xlabel='disp', ylabel='hp'>
```



In [71]:
```
# 15. Which car would I pick if I am looking for high speed as well as good fuel efficiency?
df_sorted = dat_mtcars.sort_values(by=['hp', 'mpg'], ascending=[False, False])
print(df_sorted.iloc[0])
```

```
mpg      15.00
cyl       8.00
disp    301.00
hp      335.00
drat      3.54
wt        3.57
qsec     14.60
vs        0.00
am        1.00
gear      5.00
carb      8.00
Name: Maserati Bora, dtype: float64
```

In [ ]:
```
#For the CEREALS dataset, answer the specified questions with summarization and  effective visuals.
```

In [74]:
```
cdf = pd.read_csv("Cereals1.csv")
cdf = pd.DataFrame(cdf)
n = cdf.select_dtypes(include='number').columns
for i in n:
    mean_value = cdf[i].mean()
    cdf[i].replace(-1, mean_value, inplace=True)
```

In [77]:
```
#1. How many cereals(unique brands) are there in the dataset?
x=cdf["name"].nunique()
print("Number of unique brands: ",x)
```

```
Number of unique brands:  76
```

In [89]:
```
#2. How many cereals are listed per manufacturer of the cereal?
print("Number of cereals listed per manufacturer:",cdf["mfr"].value_counts(),sep='\n')
```

```
Number of cereals listed per manufacturer:
K    23
G    22
P     9
Q     8
R     8
N     5
A     1
Name: mfr, dtype: int64
```

In [87]:
```
#3.How many hot vs cold cereals?
print("Number of cold(C) and hot(h) cereals:",cdf["type"].value_counts(),sep='\n')
```

```
Number of cold(C) and hot(h) cereals:
C    73
H     3
Name: type, dtype: int64
```

In [95]:
```
#4. Which is the best and worst cereal?
r_max=cdf["rating"].max()
print("Best cereal:")
print(cdf[cdf["rating"]==r_max]['name'])
r_min=cdf["rating"].min()
print("\nWorst cereal:")
print(cdf[cdf["rating"]==r_min]['name'])
```

```
Best cereal:
2    All-BranwithExtraFiber
Name: name, dtype: object

Worst cereal:
9    CapnCrunch
Name: name, dtype: object
```

In [105]:
```
#5. Compare the rating for hot vs cold cereals. Which do people prefer?
rating=cdf.groupby('type')['rating'].mean()
print(rating)
print("\nMost preferred: ", rating.idxmax())
```

```
type
C    41.734838
H    56.737708
Name: rating, dtype: float64

Most preferred:  H
```

In [108]:

```python
#6. Which cereals have the highest fiber and lowest sugar?
fib=cdf["fiber"].max()
print("Cereals with highest fiber:")
print(cdf[cdf["fiber"]==fib]['name'])
sug=cdf["sugars"].min()
print("\nCereals with lowest sugar:")
print(cdf[cdf["sugars"]==sug]['name'])
```

```
Cereals with highest fiber:
2    All-BranwithExtraFiber
Name: name, dtype: object

Cereals with lowest sugar:
2     All-BranwithExtraFiber
19         CreamofWheatQuick
53                PuffedRice
54               PuffedWheat
62             ShreddedWheat
63        ShreddedWheatnBran
64    ShreddedWheatspoonsize
Name: name, dtype: object
```

In [111]:

```python
#7. Which Cereals that have more than 3 grams of protein?
p = cdf[cdf['protein'] > 3]
print("Cereals having more than 3 grams of protein:")
print(p[['name', 'protein']])
```

```
Cereals with more than 3 grams of protein:
                         name  protein
1                    All-Bran        4
2       All-BranwithExtraFiber        4
10                   Cheerios        6
40                       Life        4
42                      Maypo        4
43   MuesliRaisinsDatesAlmonds        4
44  MuesliRaisinsPeachesPecans        4
55            QuakerOatSquares        4
56               QuakerOatmeal        5
66                    SpecialK        6
```

In [113]:

```python
#8
shelf_count = cdf['shelf'].value_counts()
print("\nCereals by display shelf:")
print(shelf_count)
```

```
Cereals by display shelf:
3    35
2    21
1    20
Name: shelf, dtype: int64
```

In [115]:

```python
#9
sugar = cdf.groupby('name')['sugars'].mean()
print("Average sugar content across different brands:")
print(sugar)
```

```
Average sugar content across different brands:
name
100NaturalBran             8.0
All-Bran                   5.0
All-BranwithExtraFiber     0.0
AlmondDelight              8.0
AppleCinnamonCheerios     10.0
                           ...
Triples                    3.0
Trix                      12.0
WheatChex                  3.0
Wheaties                   3.0
WheatiesHoneyGold          8.0
Name: sugars, Length: 76, dtype: float64
```

In [116]:

```
#10
avg = cdf.groupby('mfr')['calories'].mean()
print("Average calories per manufacturer:")
print(avg)
```

```
Average calories per manufacturer:
mfr
A    100.000000
G    111.363636
K    108.695652
N     90.000000
P    108.888889
Q     95.000000
R    115.000000
Name: calories, dtype: float64
```

In [118]:

```
#11
avg_nutrition = cdf[['calories', 'sugars', 'protein', 'fiber']].mean()
print("\nAverage nutritional content of cereals:")
print(avg_nutrition)
```

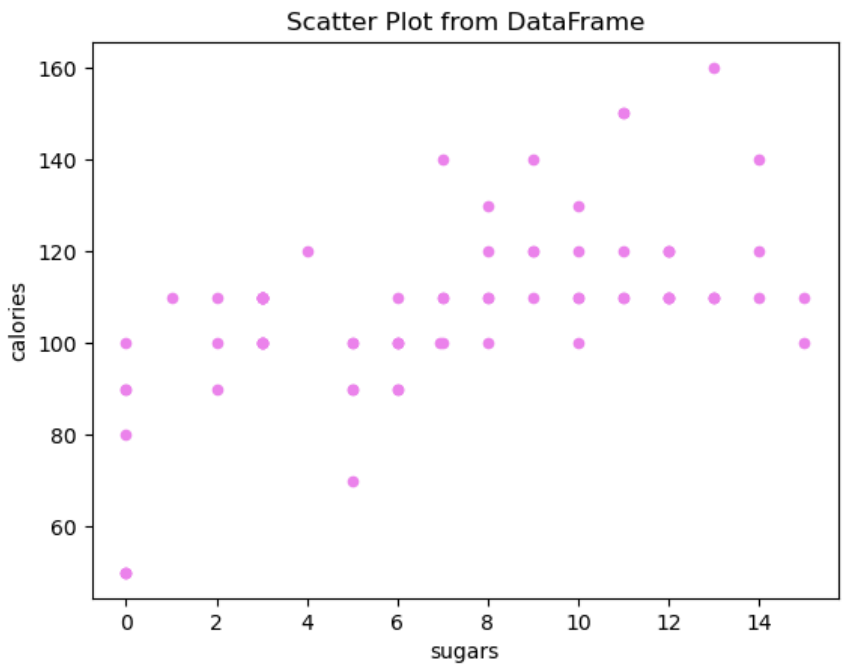
```

Average nutritional content of cereals:
calories    107.368421
sugars        7.038608
protein       2.526316
fiber         2.048684
dtype: float64
```

In [119]:

```
#12
dat_x= cdf[["sugars","calories"]]
dat_x.plot(kind='scatter', x='sugars', y='calories', color='violet', title='Scatter Plot from DataFrame'
```

Out[119]:

```
<AxesSubplot:title={'center':'Scatter Plot from DataFrame'}, xlabel='sugars', ylabel='calories'>
```



In [122]:

```
#13
high_rating_sugars = cdf[cdf['rating'] > cdf['rating'].median()]['sugars']
low_rating_sugars =cdf[cdf['rating'] <= cdf['rating'].median()]['sugars']
print("\nSugar content in cereals with high ratings:")
print(high_rating_sugars.describe())
print("\nSugar content in cereals with low ratings:")
print(low_rating_sugars.describe())
```

```
Sugar content in cereals with high ratings:
count    38.000000
mean      3.866690
std       2.884883
min       0.000000
25%       2.000000
50%       3.000000
75%       6.000000
max      12.000000
Name: sugars, dtype: float64

Sugar content in cereals with low ratings:
count    38.000000
mean     10.210526
std       3.137893
min       3.000000
25%       8.250000
50%      11.000000
75%      12.000000
max      15.000000
Name: sugars, dtype: float64
```

In [126]:

```
#14
healthy_cereals = cdf[(cdf['fiber'] > 3) & (cdf['sugars'] < 5)]
print("\nHealthy cereals with ratings:")
print(healthy_cereals[['name', 'rating', 'fiber', 'sugars']])

```

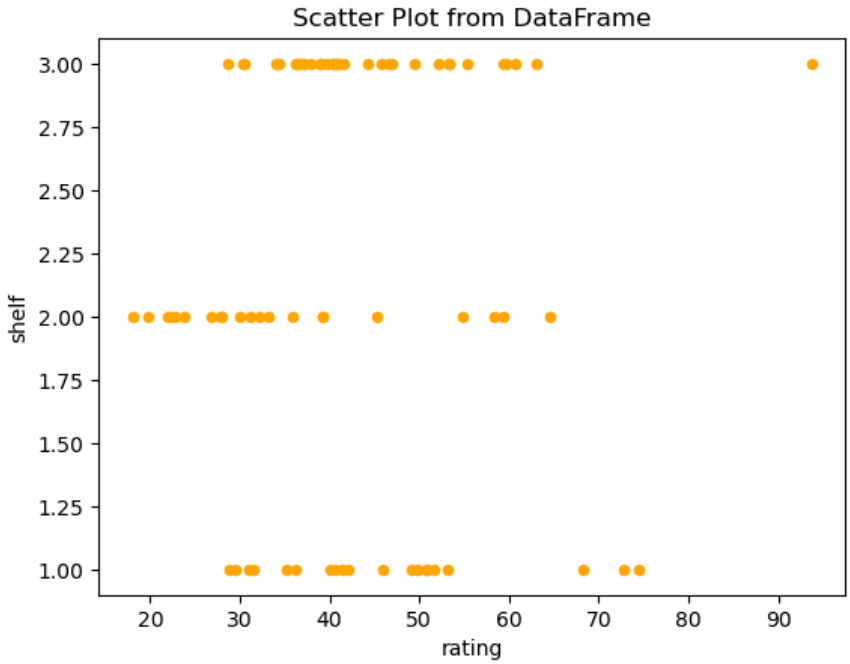
```
Healthy cereals with ratings:
                      name     rating  fiber  sugars
2   All-BranwithExtraFiber  93.704912   14.0     0.0
63      ShreddedWheatnBran  74.472949    4.0     0.0
```

In [120]:

```
#15
dat_x= cdf[["rating","shelf"]]
dat_x.plot(kind='scatter', x='rating', y='shelf', color='orange', title='Scatter Plot from DataFrame')
```

Out[120]:

```
<AxesSubplot:title={'center':'Scatter Plot from DataFrame'}, xlabel='rating', ylabel='shelf'>
```



In [ ]: